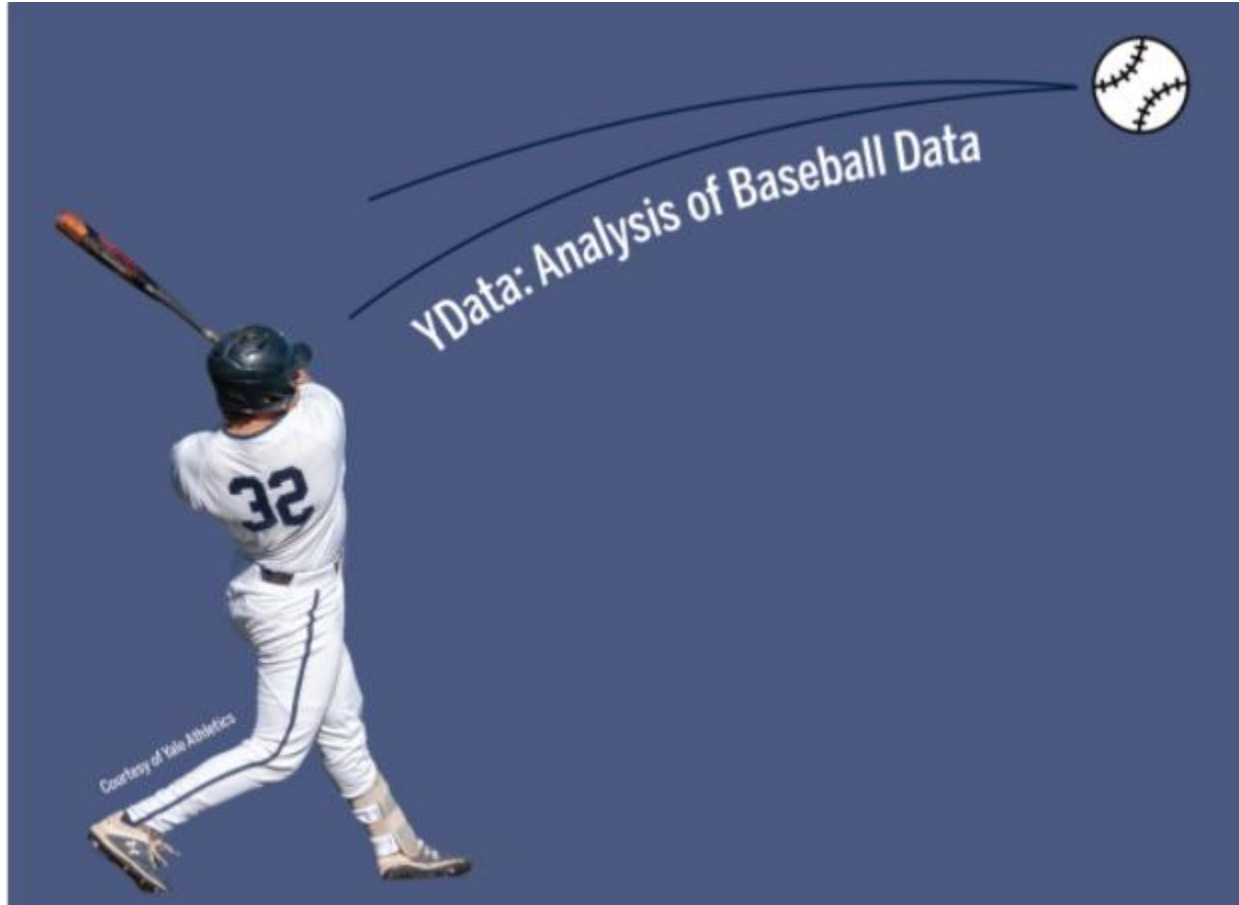


S&DS 1730



Class 2: Python basics

Overview

Brief discussion of preface and prologue to Astroball

Watch an inning of the 2025 All-star game

Review of Python Data Science code

- Data types, data structures, for loops
- Numpy ndarrays
- Pandas DataFrames

Announcement: Lab 1

Lab 1 has been released. It is due on **Wednesday January 28th**

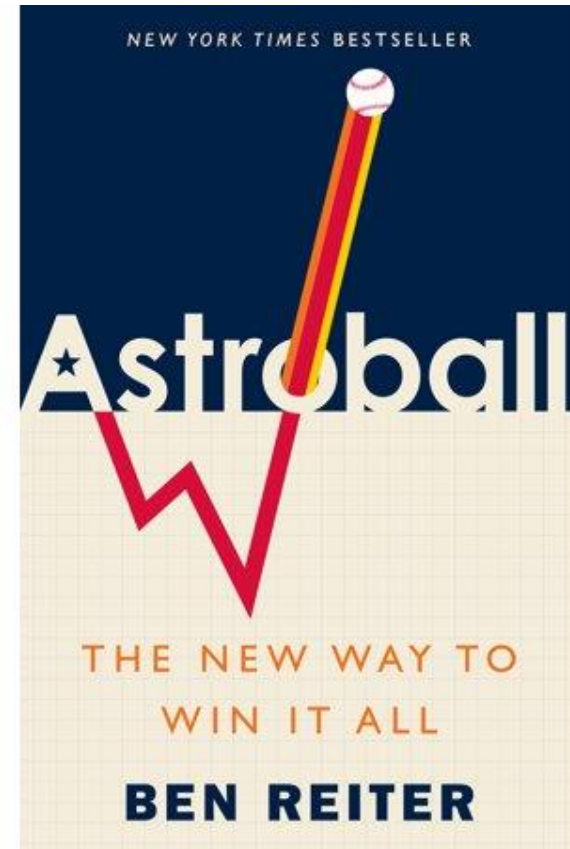
Notes:

- This will probably be the only lab this semester not due on a Sunday
- Lab 2 will be released on Monday, so there is a temporal overlap with lab 2
 - Highly recommend finishing Lab 1 by Sunday if you can

Lab 1 is short since less time to work on it

Astroball discussion of Preface and prologue

Anything about the reading that you found particular interesting?



Preface and prologue of Astrobball?

Season	League	Division	Finish[2]	Wins[2]	Losses[2]	Win% ^[2]	GB ^[2]
2010	NL	Central	4th	76	86	0.469	15
2011	NL	Central	6th	56	106	0.346	37½
2012	NL	Central	6th	55	107	0.340	42
2013	AL	West	5th	51	111	0.315	45
2014	AL	West	4th	70	92	0.432	28
2015	AL	West	2nd ꝥ	86	76	0.531	2

[Jeopardy 11/18/2013](#)

Preface and prologue of Astrobball?



[Astrodome](#)

Baseball basics



To learn the basics of baseball let's watch and keep score for the [2025 All-Star game](#)

Scorecard

2014 MLB All-Star Game

Score: AL 5, NL 3

AMERICAN LEAGUE		1	2	3	4	5	6	7	8	9	10	11	12	Runs	Hits	Errors	Left On
Player	Score																
Derek Jeter SS	1	1												0	2	1	19
Mike Trout LF	TRP	1												2	1	1	10
Mike Trout LF	RBI	1												4	1	1	11
David Kohn C	2													3	3	0	10
Cabrera Byes	K													0	1	0	5
Tammy Oators	K													2	1	0	2
Jarin Hroat RS	K													1	1	0	4
Miguel Cabrera 1B	HR													1	0	0	2
Runs	3	2	1	1	0	2	1	0						Final	4-7	15	108
Hits	11	1	1	1	1	2	0	1									
Errors	0	0	3	0	1	0	1	0									
Left On	1	2	3	1	1	0	2	1									

NATIONAL LEAGUE		1	2	3	4	5	6	7	8	9	10	11	12	Runs	Hits	Errors	Left On
Player	Team																
Andrew McCutchen	CF	1												4	7	2	7
Yasiel Puig	RF	1												3	3	0	4
Marcy Buids	DF	1												12	3	0	0
Josep Hames	PK	1												5	1	4	2
Chase Utley	2B	1												4	1	0	4
Jerens Willson	C	1												15	2	1	0
Jonathan Lucroy	C	1												2	1	0	1
Runs	1	0	1	0	2	2	2	2						Final Score			
Hits	3	2	2	0	1	1	3							AL 5, NL 3			
Errors	1	1	1	0	1	0	1										
Left On	3	2	0	0	1	0	0										

[More official score keeping](#)

Let's fill in a scorecard as we watch...

[illegible]

2025 All-star game

American					National			
Order	Player	Team	Position		Order	Player	Team	Position
1	Gleyber Torres	Tigers	2B		1	Shohei Ohtani	Dodgers	DH
2	Riley Greene	Tigers	LF		2	Ronald Acuña Jr.	Braves	LF
3	Aaron Judge	Yankees	RF		3	Ketel Marte	Diamondbacks	2B
4	Cal Raleigh	Mariners	C		4	Freddie Freeman	Dodgers	1B
5	Vladimir Guerrero Jr.	Blue Jays	1B		5	Manny Machado	Padres	3B
6	Ryan O'Hearn	Orioles	DH		6	Will Smith	Dodgers	C
7	Junior Caminero	Rays	3B		7	Kyle Tucker	Cubs	RF
8	Javier Báez	Tigers	CF		8	Francisco Lindor	Mets	SS
9	Jacob Wilson	Athletics	SS		9	Pete Crow-Armstrong	Cubs	CF
—	Tarik Skubal	Tigers	P		—	Paul Skenes	Pirates	P



Let's watch an inning of the [game](#)

Retrosheet play-by-play data

	gid	inning	top_bot	outs_pre	score_v	score_h	batteam	batter	pitcher	pitches	event
0	NLS202507150	1	0	0	0	0	ALS	torrg001	skenp001	FBTBS	K
1	NLS202507150	1	0	1	0	0	ALS	greer003	skenp001	BCCBFBS	K
2	NLS202507150	1	0	2	0	0	ALS	judga001	skenp001	CX	43/G4M
3	NLS202507150	1	1	0	0	0	NLS	ohtas001	skubt001	FFBX	S8/L8
4	NLS202507150	1	1	0	0	0	NLS	acunr001	skubt001	FSBX	S4/G4S-.1-2
5	NLS202507150	1	1	0	0	0	NLS	martk001	skubt001	SX	D9/G3+.2-H;1-H
6	NLS202507150	1	1	0	0	2	NLS	freef001	skubt001	BSX	53/G56
7	NLS202507150	1	1	1	0	2	NLS	machm001	skubt001	FSC	K
8	NLS202507150	1	1	2	0	2	NLS	smitw003	skubt001	CTS	K
9	NLS202507150	2	0	0	0	2	ALS	ralec001	kersc001	.X	7/L7LD+

Interpreting each pitch event (pitches column)

<https://www.retrosheet.org/eventfile.htm#5>

Structured data

Variables

Cases

	gid	inning	top_bot	outs_pre	score_v	score_h	batteam	batter	pitcher	pitches	event
0	NLS202507150	1	0	0	0	0	ALS	torrg001	skenp001	FBTBS	K
1	NLS202507150	1	0	1	0	0	ALS	greer003	skenp001	BCCBFBS	K
2	NLS202507150	1	0	2	0	0	ALS	judga001	skenp001	CX	43/G4M
3	NLS202507150	1	1	0	0	0	NLS	ohtas001	skubt001	FFBX	S8/L8
4	NLS202507150	1	1	0	0	0	NLS	acunr001	skubt001	FSBX	S4/G4S-.1-2
5	NLS202507150	1	1	0	0	0	NLS	martk001	skubt001	SX	D9/G3+.2-H;1-H
6	NLS202507150	1	1	0	0	2	NLS	freef001	skubt001	BSX	53/G56
7	NLS202507150	1	1	1	0	2	NLS	machm001	skubt001	FSC	K
8	NLS202507150	1	1	2	0	2	NLS	smitw003	skubt001	CTS	K
9	NLS202507150	2	0	0	0	2	ALS	ralec001	kersc001	.X	7/L7LD+

Structured data

Quantitative Variable

Categorical Variable

Cases

	gid	inning	top_bot	outs_pre	score_v	score_h	batteam	batter	pitcher	pitches	event
0	NLS202507150	1	0	0	0	0	ALS	torrg001	skenp001	FBTBS	K
1	NLS202507150	1	0	1	0	0	ALS	greer003	skenp001	BCCBFBS	K
2	NLS202507150	1	0	2	0	0	ALS	judga001	skenp001	CX	43/G4M
3	NLS202507150	1	1	0	0	0	NLS	ohtas001	skubt001	FFBX	S8/L8
4	NLS202507150	1	1	0	0	0	NLS	acunr001	skubt001	FSBX	S4/G4S-.1-2
5	NLS202507150	1	1	0	0	0	NLS	martk001	skubt001	SX	D9/G3+.2-H;1-H
6	NLS202507150	1	1	0	0	2	NLS	freef001	skubt001	BSX	53/G56
7	NLS202507150	1	1	1	0	2	NLS	machm001	skubt001	FSC	K
8	NLS202507150	1	1	2	0	2	NLS	smitw003	skubt001	CTS	K
9	NLS202507150	2	0	0	0	2	ALS	ralec001	persc001	.X	7/L7LD+

Retrosheet play-by-play data

Let's take a very quick look at the retrosheet play-by-play data in Python

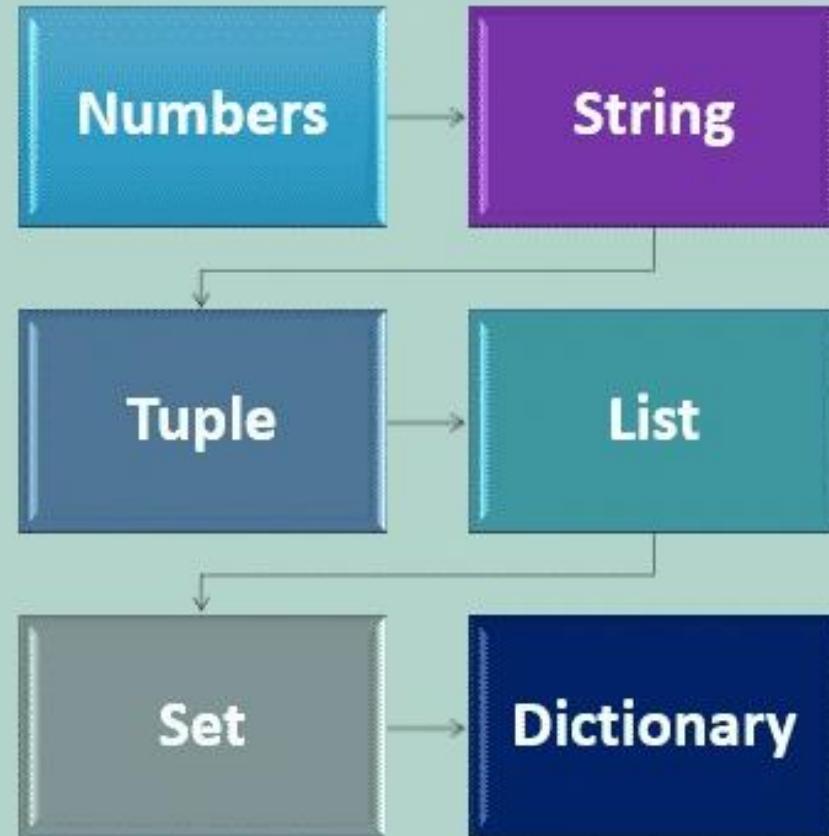
Please download class 2 code:

```
import YData_baseball  
YData_baseball.download_class_code(2)
```

You will do some exercises on this data in lab 1

Python

Data Types



Python basic data types

Some of the fundamental data types in Python are:

Integer (int): Represents whole numbers

Float (float): Represents real numbers

String (str): Represents sequences of characters, used for text.

- Strings are created by enclosing characters in single (' '), double (" ")

Boolean (bool): Represents truth values, which can only be **True** or **False**

Let's explore these in Jupyter!

Python basic structures

Some of the fundamental data structures in Python are:

Lists: An ordered and **mutable** collection of items.

- Lists can contain elements of different data types and are defined using square brackets []
- Example: `x = ["apple", "banana", "cherry"]`

Tuples: Similar to lists, but tuples are **immutable** (cannot be changed after creation) and are defined using round brackets ().

- Example: `x = ("apple", "banana", "cherry")`

Dictionaries: An unordered collection of data stored as key-value pairs.

- Dictionaries are mutable and defined using curly braces {}.
- Example: `x = {"name" : "John", "age" : 36}`

Let's explore these in Jupyter!

Sequences and for loops

For loops repeat a process many times, iterating over a sequence of items

- Often we are iterating over sequential integers

```
lineup = ["Torres", "Greene", "Judge"]
```

```
for player in lineup:  
    print(player)
```

```
for i in range(3):  
    print(i**2)
```

Examples of sequences:

- Lists, ndarrays, characters in strings

Let's explore these in Jupyter!

Array computations using numpy



The *NumPy package* efficiently stores and processes data that is all of the same type using *ndarray*

```
import numpy as np  
  
at_bats = np.array([326, 384, 153])      # creating an ndarray  
walks = np.array([37, 32, 22])
```

We can easily do element wise computations using arrays

```
plate_appearances = at_bats + walks      # + hit_by_pitch + intentional_walks
```

There are also a number of numpy functions that operate efficiently on ndarrays

```
np.sum(), np.mean(), np.diff(), etc.
```

Let's explore these in Jupyter!

Boolean arrays

It is often to compare all values in an ndarray to a particular value

```
my_array = np.array([12, 4, 3])
```

```
my_array < 5      # Q: What does this return?      A: array([False, True, True])
```

We can also use "Boolean Indexing" to get a subset of values

```
my_array2 = np.array([12, 1, 6,])
```

```
boolean_mask = np.array([False, True, True])
```

```
smaller_array = my_array2[boolean_mask]      A: array([1, 6])
```

This can be useful for getting data that meet particular criteria:

```
my_array[my_array < 5]      # what does this do?      A: array([4, 3])
```

Let's explore these in Jupyter!

Pandas DataFrames



Pandas: Series and DataFrames

[Pandas](#) is a package that is useful for processing time series and tables of data

There are two main data structures in pandas:

- **Series:** represent one-dimensional data
- **DataFrames:** represent data tables
 - i.e., relational data



pandas Series

Series are one-dimensional ndarrays with index names

yearID	
2021	46
2022	34
2023	44
2024	54

Index

array of values

Often, we treat Series similar to numpy ndarrays:

- E.g., we do things like: `np.sum(my_series)`

pandas DataFrames

Pandas DataFrame hold Table data

This is one of the most useful formats to extract insights from datasets

Pandas DataFrames have an Index, and columns of data

The Index here is just sequential numbers but we could set it to be the playerId or values in another column



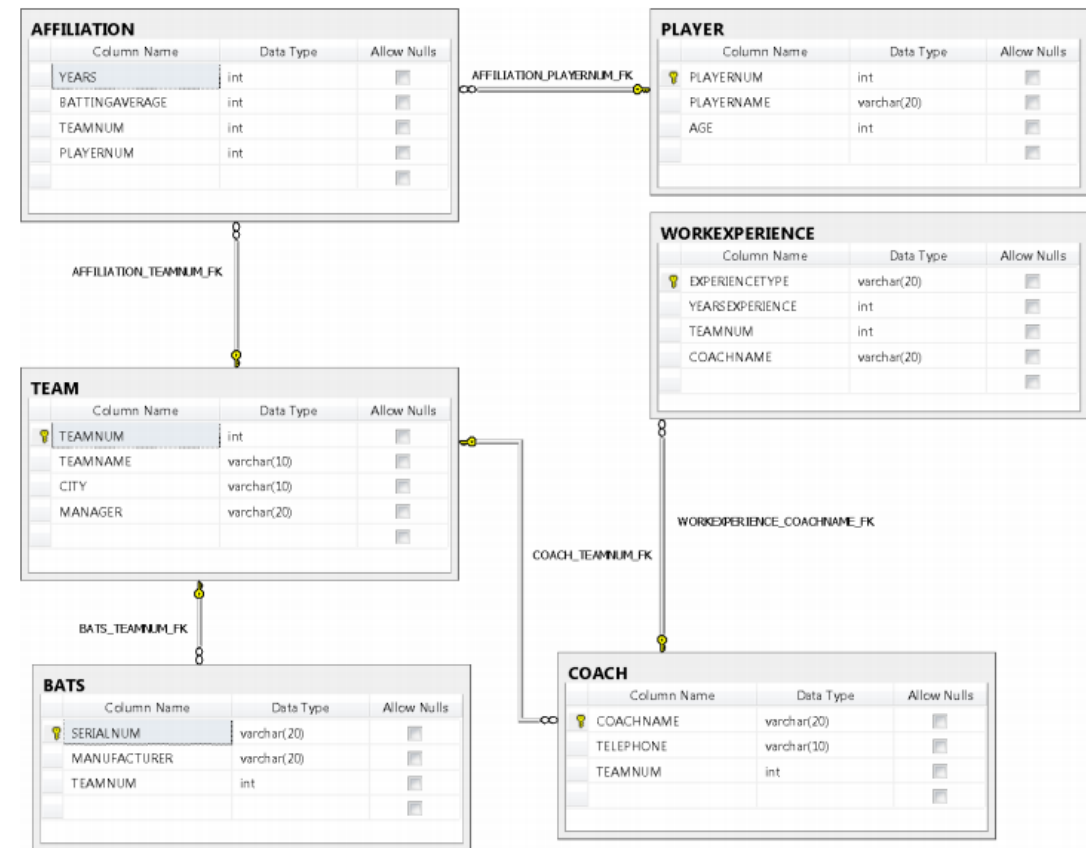
	playerID	yearID	stint	teamID	lgID	G	G_batting	AB	R	H
1	aardsda01	2004	1	SFN	NL	11	11	0	0	0
2	aardsda01	2006	1	CHN	NL	45	43	2	0	0
3	aardsda01	2007	1	CHA	AL	25	2	0	0	0
4	aardsda01	2008	1	BOS	AL	47	5	1	0	0
5	aardsda01	2009	1	SEA	AL	73	3	0	0	0
6	aardsda01	2010	1	SEA	AL	53	4	0	0	0
7	aardsda01	2012	1	NYA	AL	1	NA	NA	NA	NA
8	aaronha01	1954	1	ML1	NL	122	122	468	58	131

Lahman Database: Season level statistics

Lahman database contains season level information about Major League baseball players from 1871 to 2024

```
import pandas as pd
```

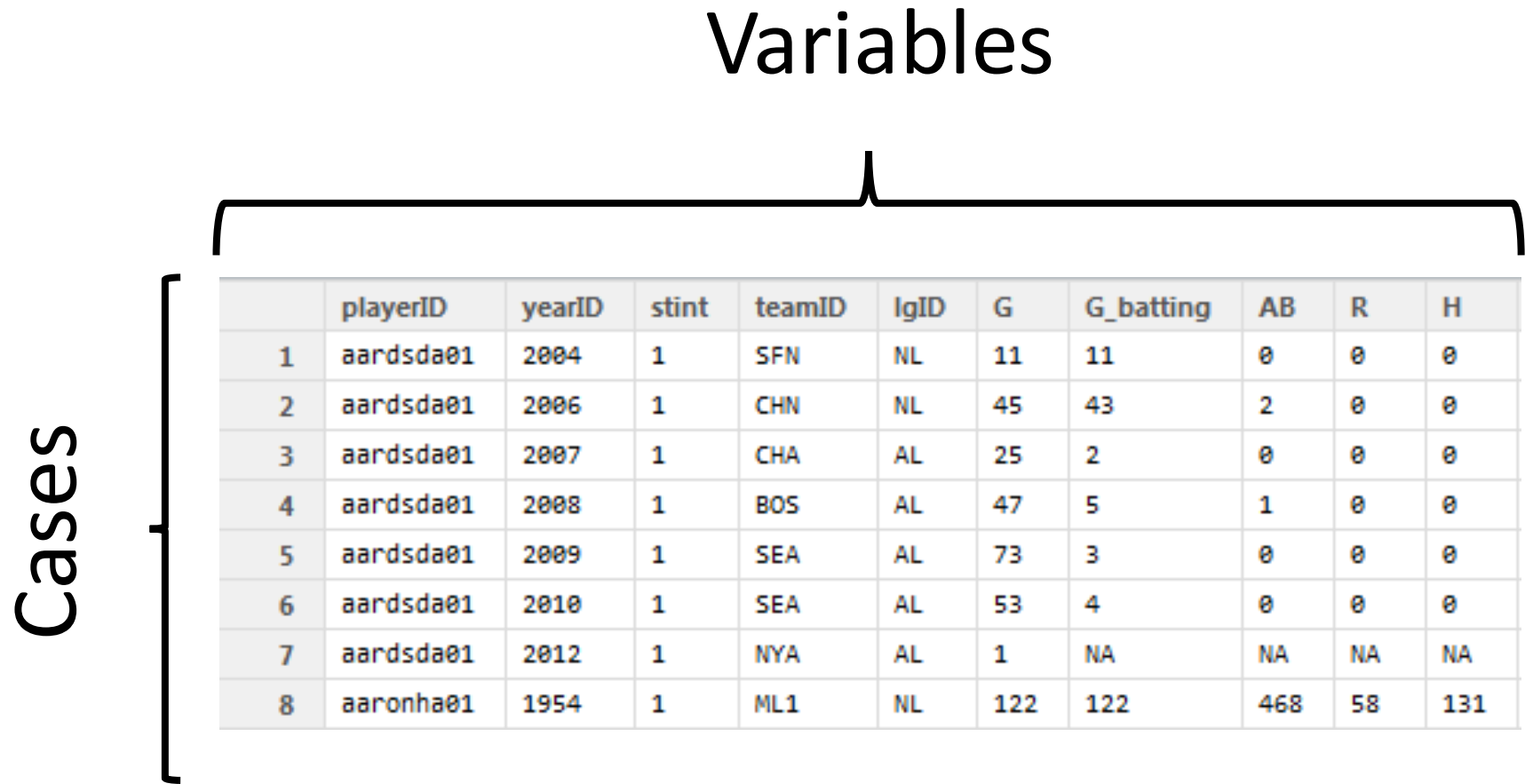
```
batting = pd.read_csv('Batting.csv')
```



Structured data: Batting dataset

Variables

Cases



A diagram illustrating the structure of a dataset. A large horizontal curly bracket is positioned above the table, with the word 'Variables' centered above it. A large vertical curly bracket is positioned to the left of the table, with the word 'Cases' centered next to it.

	playerID	yearID	stint	teamID	lgID	G	G_batting	AB	R	H
1	aardsda01	2004	1	SFN	NL	11	11	0	0	0
2	aardsda01	2006	1	CHN	NL	45	43	2	0	0
3	aardsda01	2007	1	CHA	AL	25	2	0	0	0
4	aardsda01	2008	1	BOS	AL	47	5	1	0	0
5	aardsda01	2009	1	SEA	AL	73	3	0	0	0
6	aardsda01	2010	1	SEA	AL	53	4	0	0	0
7	aardsda01	2012	1	NYA	AL	1	NA	NA	NA	NA
8	aaronha01	1954	1	ML1	NL	122	122	468	58	131

Structured data

Variables

Cases

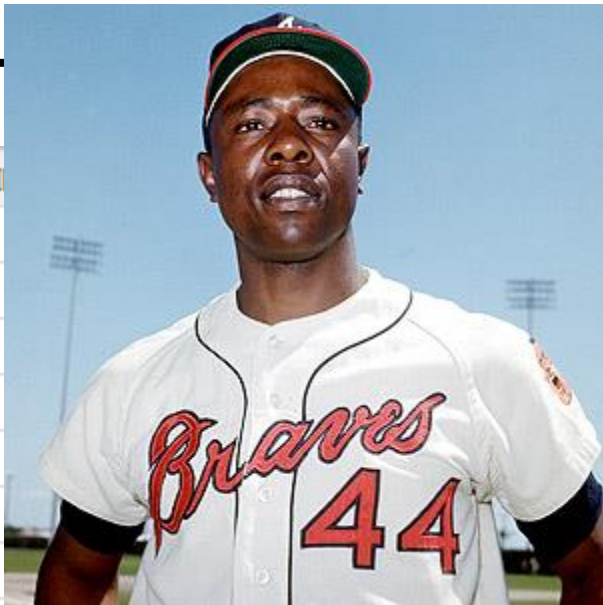
	playerID	yearID	stint	teamID	lgID	G	G_batting	AB	R	H
1	aardsda01	2004	1					0	0	0
2	aardsda01	2006	1					2	0	0
3	aardsda01	2007	1					0	0	0
4	aardsda01	2008	1					1	0	0
5	aardsda01	2009	1					0	0	0
6	aardsda01	2010	1					0	0	0
7	aardsda01	2012	1					NA	NA	NA
8	aaronha01	1954	1	ML1	NL	122	122	468	58	131

Structured data

Variables

Cases

	playerID	yearID	stint	teamID	lgID					
1	aardsda01	2004	1	SFN	NL					
2	aardsda01	2006	1	CHN	NL					
3	aardsda01	2007	1	CHA	AL					
4	aardsda01	2008	1	BOS	AL					
5	aardsda01	2009	1	SEA	AL					
6	aardsda01	2010	1	SEA	AL					
7	aardsda01	2012	1	NYA	AL					
8	aaronha01	1954	1	ML1	NL	122	122	468	58	131



Categorical and Quantitative Variables

Categorical Variable

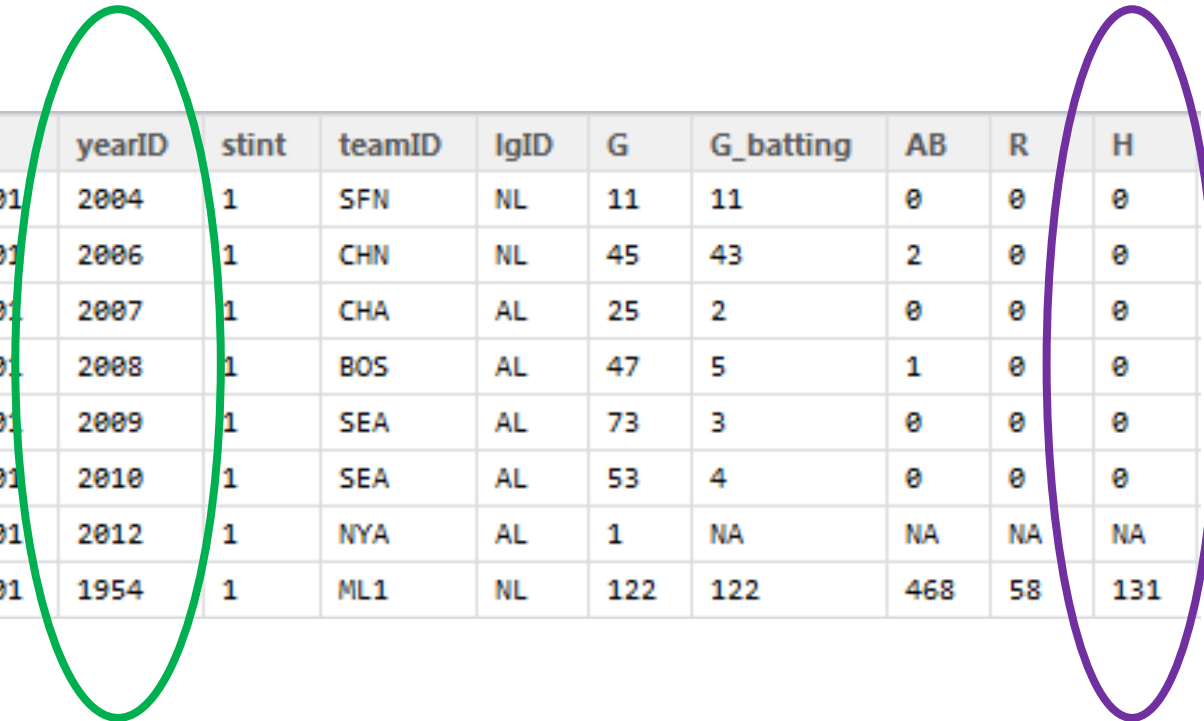
Quantitative Variable

Cases

	playerID	yearID	stint	teamID	lgID	G	G_batting	AB	R	H
1	aardsda01	2004	1	SFN	NL	11	11	0	0	0
2	aardsda01	2006	1	CHN	NL	45	43	2	0	0
3	aardsda01	2007	1	CHA	AL	25	2	0	0	0
4	aardsda01	2008	1	BOS	AL	47	5	1	0	0
5	aardsda01	2009	1	SEA	AL	73	3	0	0	0
6	aardsda01	2010	1	SEA	AL	53	4	0	0	0
7	aardsda01	2012	1	NYA	AL	1	NA	NA	NA	NA
8	aaronha01	1954	1	ML1	NL	122	122	468	58	131

Explanatory and Response Variables

Sometimes we use one variable (**the explanatory variable**) to understand/predict another variable (**the response variable**)

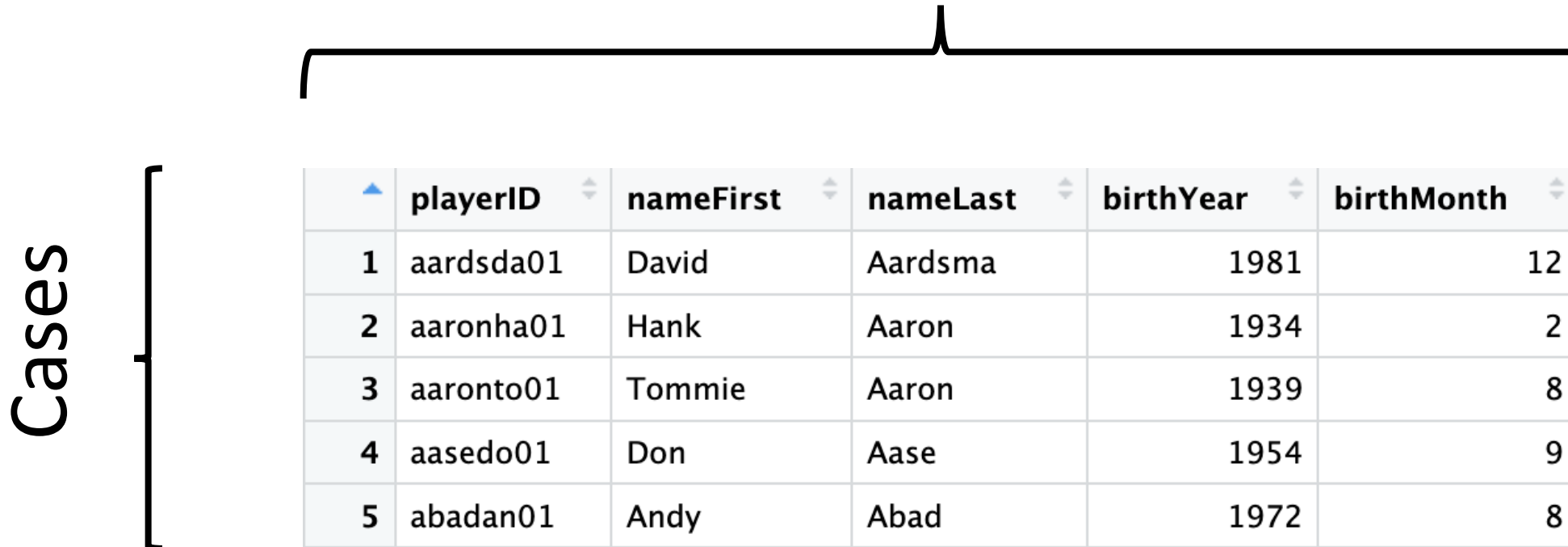


	playerID	yearID	stint	teamID	lgID	G	G_batting	AB	R	H
1	aardsda01	2004	1	SFN	NL	11	11	0	0	0
2	aardsda01	2006	1	CHN	NL	45	43	2	0	0
3	aardsda01	2007	1	CHA	AL	25	2	0	0	0
4	aardsda01	2008	1	BOS	AL	47	5	1	0	0
5	aardsda01	2009	1	SEA	AL	73	3	0	0	0
6	aardsda01	2010	1	SEA	AL	53	4	0	0	0
7	aardsda01	2012	1	NYA	AL	1	NA	NA	NA	NA
8	aaronha01	1954	1	ML1	NL	122	122	468	58	131

People Dataset: Information on players

Variables

Cases

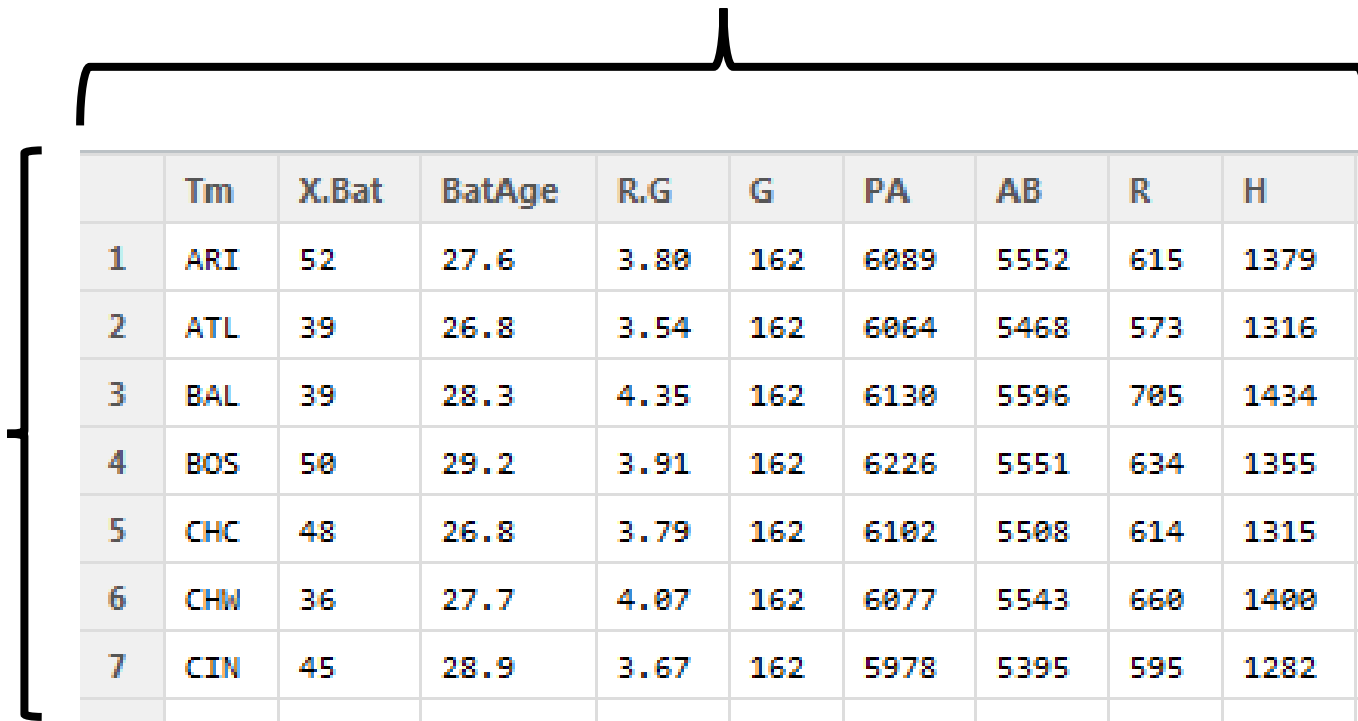


	playerID	nameFirst	nameLast	birthYear	birthMonth
1	aardsda01	David	Aardsma	1981	12
2	aaronha01	Hank	Aaron	1934	2
3	aaronto01	Tommie	Aaron	1939	8
4	aasedo01	Don	Aase	1954	9
5	abadan01	Andy	Abad	1972	8

Another Dataset: Team statistics

Variables

Cases



	Tm	X.Bat	BatAge	R.G	G	PA	AB	R	H
1	ARI	52	27.6	3.80	162	6089	5552	615	1379
2	ATL	39	26.8	3.54	162	6064	5468	573	1316
3	BAL	39	28.3	4.35	162	6130	5596	705	1434
4	BOS	50	29.2	3.91	162	6226	5551	634	1355
5	CHC	48	26.8	3.79	162	6102	5508	614	1315
6	CHW	36	27.7	4.07	162	6077	5543	660	1400
7	CIN	45	28.9	3.67	162	5978	5395	595	1282
8	CLE	43	28.5	4.13	162	6222	5575	669	1411

Loading the Lahman data into Python

We can use `pd.read_csv()` to read in .csv files as pandas DataFrames

```
import pandas as pd
```

```
batting = pd.read_csv('Batting.csv')
```

```
players = pd.read_csv("People.csv")
```

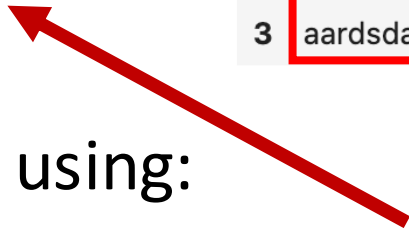
Pandas DataFrame operations

Selecting a subset of columns

1. We can select a subset of columns using:

```
my_df[["col1", "col2"]]
```

	playerID	yearID	stint	teamID	lgID	G
0	aardsda01	2004	1	SFN	NL	11
1	aardsda01	2006	1	CHN	NL	45
2	aardsda01	2007	1	CHA	AL	25
3	aardsda01	2008	1	BOS	AL	47



2. We can select a single column as a **Series** using:

```
my_df["col1"]
```

Useful to get a more manageable dataset

We can then apply numpy operations on the data.
E.g.,

```
total_stat = np.sum(my_df["col1"])
```

```
derived_stat = my_df["col1"] + my_df["col1"]
```

Useful to add new columns to a dataset



Let's explore these in Jupyter!

Selecting a subset of rows based on position

We can select a subset of rows based on position using `.iloc`:

```
my_df.iloc[[0, 10, 20]]    # since given a list of indices, returns a DataFrame
```

This is often useful for getting the first or last row:

```
my_df.iloc[0]    # since given number as an index, returns a Series  
my_df.iloc[-1]   # last row
```

We can also select rows based on Index values using `.loc`

```
batting2 = batting.set_index()
```

Let's explore these in Jupyter!

Sorting rows of a DataFrame

We can sort the rows of a DataFrame using:

```
my_df.sort_values("col_name")
```

We can sort the rows from largest to smallest value using:

```
my_df.sort_values("col_name", ascending = False)
```

Let's explore these in Jupyter!

Getting a subset of rows using Boolean indexing

We can get a subset of rows using that meet particular criteria using Boolean indexing:

```
my_df[my_df["col_name"] == 7] # getting rows using a Boolean mask
```

We can also do this (in many cases) using the .query() method

```
my_df.query("col_name == 7") # getting rows using the query function
```

Let's explore these in Jupyter!

Adding and renaming columns

We can add an ndarray (of the appropriate length) as a column to a DataFrame using:

```
my_df["new_col"] = col_values
```

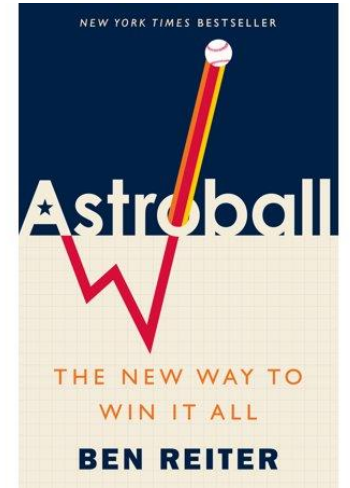
We can rename columns using the .rename() method:

```
rename_dictionary = {"old_col_name": "new_col_name"}  
my_df.rename(columns = rename_dictionary )
```

Let's explore these in Jupyter!

For next class

1. Read chapter 1 of Astrobball
2. If you have not done this yet: [fill out the background survey](#)
3. Try to complete lab 1
 - Although not due until Wednesday, so you a little more time if you get stuck



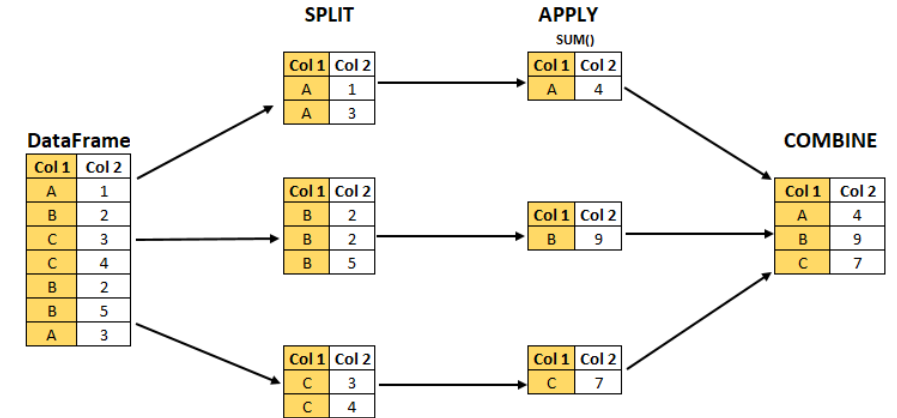
If there is more time....

Creating aggregate statistics by group

We can get statistics separately by group:

```
baseball.groupby("Year").agg("max")
```

```
my_df.groupby("group_col_name").agg(  
    new_col1 = ('col_name', 'statistic_name1'),  
    new_col2 = ('col_name', 'statistic_name2'),  
    new_col3 = ('col_name', 'statistic_name3')  
)
```



```
batting.groupby("teamID").agg(  
    max_HR = ("HR", "max"),  
    first_player = ("playerID", "min")  
)
```

Let's explore these in Jupyter!

Merging/Joining DataFrames

The DataFrames in the Lahman database can be connected through different ID columns

- For example, we can relate the Batting and People DataFrames using the playerID column

Batting DataFrame

	playerID	yearID	stint	teamID	lgID	G	G_batting	AB	R	H
1	aardsda01	2004	1	SFN	NL	11	11	0	0	0
2	aardsda01	2006	1	CHN	NL	45	43	2	0	0
3	aardsda01	2007	1	CHA	AL	25	2	0	0	0
4	aardsda01	2008	1	BOS	AL	47	5	1	0	0
5	aardsda01	2009	1	SEA	AL	73	3	0	0	0
6	aardsda01	2010	1	SEA	AL	53	4	0	0	0
7	aardsda01	2012	1	NYA	AL	1	NA	NA	NA	NA
8	aaronha01	1954	1	ML1	NL	122	122	468	58	131

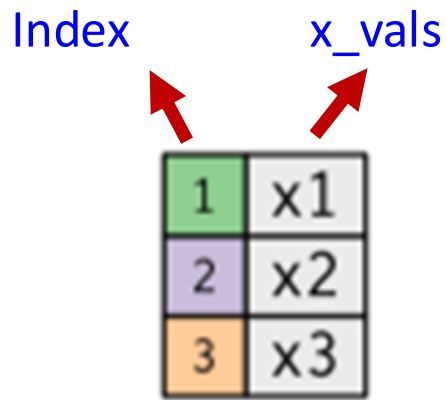
People DataFrame

	playerID	nameFirst	nameLast	birthYear	birthMonth
1	aardsda01	David	Aardsma	1981	12
2	aaronha01	Hank	Aaron	1934	2
3	aaronto01	Tommie	Aaron	1939	8
4	aasedo01	Don	Aase	1954	9
5	abadan01	Andy	Abad	1972	8

Joining

Suppose we have two DataFrames (or Series) called **x_df** and **y_df**

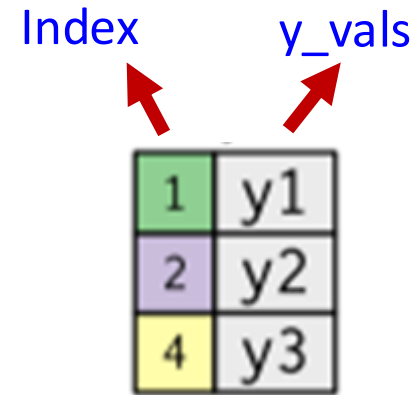
- **x_df** have one column called **x_vals**
- **y_df** has one column called **y_vals**



A diagram of a DataFrame with two columns. The first column is labeled 'Index' with a red arrow pointing to it. The second column is labeled 'x_vals' with a red arrow pointing to it. The DataFrame contains three rows: the first row has index 1 and value x1; the second row has index 2 and value x2; the third row has index 3 and value x3.

1	x1
2	x2
3	x3

DataFrame: x_df



A diagram of a DataFrame with two columns. The first column is labeled 'Index' with a red arrow pointing to it. The second column is labeled 'y_vals' with a red arrow pointing to it. The DataFrame contains three rows: the first row has index 1 and value y1; the second row has index 2 and value y2; the third row has index 4 and value y3.

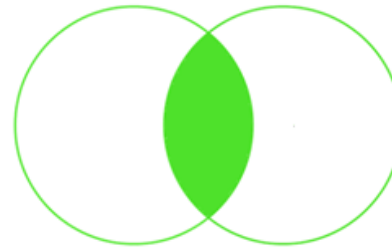
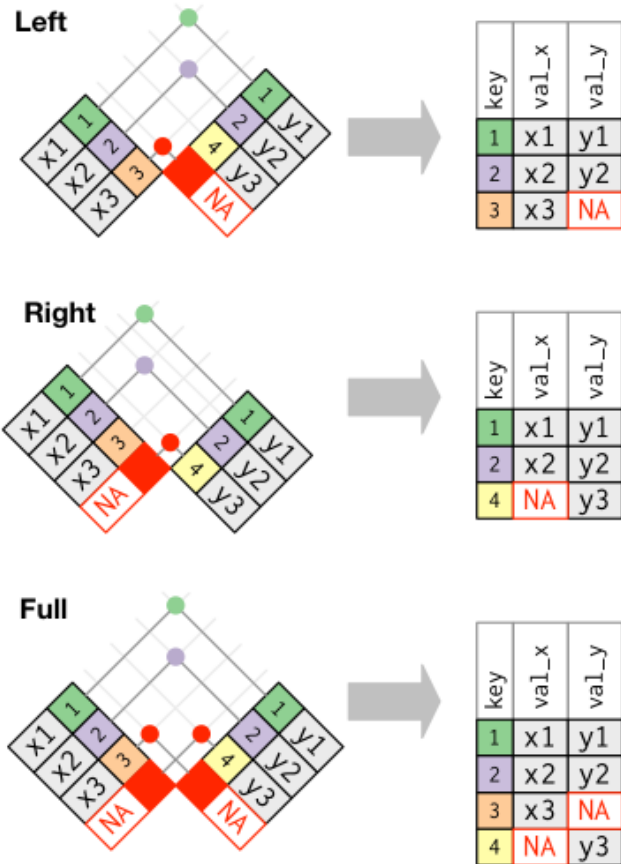
1	y1
2	y2
4	y3

DataFrame: y_df

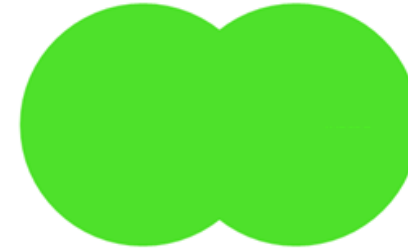
We can join these two DataFrames into a single DataFrame by aligning rows with the same Index value using the general syntax: **x_df.join(y_df)**

- i.e., the new joined data frame will have two columns: **x_vals**, and **y_vals**

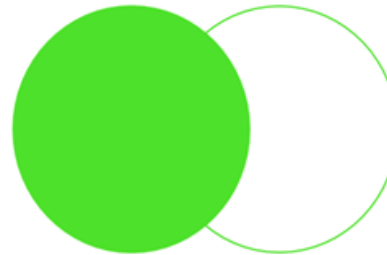
Joining



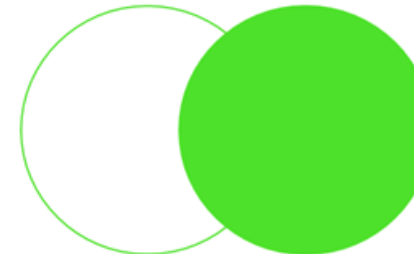
INNER JOIN



FULL OUTER JOIN



LEFT JOIN



RIGHT JOIN

`x_df.join(y_df, how = "left")` # based on Index

`x_df.merge(y_df, how = "inner", left_on = "x_col", right_on = "y_col")` # based on columns